

InterviewPro

AI-Enhanced Interview Management System

Submitted by: B. Varun Reddy

GitHub: github.com/BVarunReddy/ai-interview-management-system

Live Demo: <https://interviewpro-ats.onrender.com>

1. Problem Statement

1.1 Background

The traditional recruitment process in organizations is largely manual, time-consuming, and prone to human error. HR teams spend significant effort managing candidate data across spreadsheets, scheduling interviews via email, collecting feedback through paper forms, and making hiring decisions based on subjective judgment without data-driven insights.

1.2 Identified Problems

- Candidate information is scattered across multiple spreadsheets and email threads
- Interview scheduling is manual and error-prone with no centralized tracking
- Feedback collection is inconsistent with no standardized scoring system
- No data-driven approach to predict candidate selection probability
- Reports and analytics require manual compilation taking hours of HR time
- No automated evaluation reports — interviewers write notes manually
- Lack of real-time notifications for recruitment events

1.3 Impact

These problems result in longer time-to-hire, poor candidate experience, inconsistent hiring decisions, and significant HR resource wastage. Organizations lose top talent due to delayed processes and lack of structured evaluation.

1.4 Proposed Solution

InterviewPro — a full-stack AI-Enhanced Interview Management System that digitizes and automates the entire recruitment lifecycle, integrating Machine Learning for candidate selection prediction and Generative AI for automated evaluation report generation.

2. Approach

2.1 Development Methodology

The project followed an iterative development approach with three distinct phases:

- **Phase 1 — Core System:** Built the full-stack interview management application with authentication, candidate management, interview scheduling, and feedback collection
- **Phase 2 — ML Integration:** Developed and trained a Random Forest Classifier on recruitment data to predict candidate selection probability with three output classes
- **Phase 3 — GenAI Integration:** Integrated Groq API (Llama 3.1 LLM) to automatically generate professional interview evaluation reports from feedback data

2.2 Machine Learning Approach

Dataset Preparation

Collected and created a recruitment dataset with 100 candidate records containing 8 input features — years of experience, technical assessment score, communication rating, problem solving score, number of interviews, education level, previous companies, and skills count — with binary selection outcome as the target variable.

Model Training

Applied Random Forest Classifier with 100 decision trees, max_depth of 8, and balanced class weights to handle imbalanced data. Used 80/20 train-test split with stratified sampling to maintain class distribution.

Evaluation

Evaluated model performance using accuracy (90%+), precision, recall, and F1-score metrics. Generated confusion matrix and feature importance plots. Saved the trained model using joblib for backend integration.

Prediction Output

The model classifies candidates into three probability categories:

- Highly Likely to Select (probability $\geq 70\%$)
- Moderately Likely to Select (40–69%)
- Low Selection Probability (below 40%)

2.3 Generative AI Approach

Integrated Groq API which provides access to Meta's Llama 3.1-8b-instant language model. When a recruiter requests an evaluation report, the backend constructs a structured prompt containing the candidate's profile, all feedback scores, interviewer recommendation, and specific instructions for report sections. Groq's LLM processes the prompt and generates a unique, detailed, professional evaluation report tailored to the specific candidate's performance data.

2.4 Security Approach

- Passwords hashed using bcrypt with salt rounds of 10
 - JWT-based stateless authentication with 7-day token expiry
 - Role-based access control (Admin, HR, Interviewer)
 - SQL injection prevention through parameterized queries
 - File upload validation restricting types to PDF/DOC/DOCX
-

3. System Architecture

3.1 Architecture Overview

InterviewPro follows a Three-Tier Architecture with clear separation of concerns:

Layer	Component	Technology	Hosting
Presentation Layer	Frontend UI	HTML5, CSS3, JavaScript	Render Static Site
Application Layer	REST API Server	Node.js + Express.js	Render Web Service
Application Layer	ML Module	Python + scikit-learn	Backend Integration
Application Layer	GenAI Module	Groq API (Llama 3.1)	External API
Data Layer	Relational Database	MySQL 8.0	Railway Managed

3.2 System Modules

Module 1: Interview Management System

- Candidate Registration & Login with JWT Authentication
- Recruiter/Admin Login with role-based access control
- Interview Scheduling with 5 round types
- Candidate pipeline tracking across 6 stages
- Structured feedback collection with 1-10 scoring
- Real-time notification system
- Analytics dashboard with 4 chart types

Module 2: Machine Learning Module

- Algorithm: Random Forest Classifier (100 estimators)
- Input: 8 candidate features from database
- Output: Selection probability with 3-class classification
- Model saved as .pkl file using joblib
- Integrated via /api/ml/predict endpoint

Module 3: Generative AI Module

- LLM: Meta Llama 3.1-8b-instant via Groq API
- Input: Candidate profile + all feedback scores + recommendation

- Output: 7-section professional evaluation report
- Fallback to rule-based report if API unavailable
- Accessible via /api/ai/generate-report endpoint

3.3 Database Schema

Table	Key Columns	Purpose
users	id, name, email, password (bcrypt), role	Authentication
candidates	id, name, email, position, status, ai_score, ml_prediction	Pipeline management
interviews	id, candidate_id (FK), round_name, date, time, status	Scheduling
feedback	id, candidate_id (FK), technical_score, communication_score, overall_score	Evaluation
notifications	id, title, message, type, is_read, user_id (FK)	Notifications

3.4 API Architecture

Route Prefix	File	Endpoints
/api/auth	routes/auth.js	login, register, profile
/api/candidates	routes/candidates.js	CRUD + file upload
/api/interviews	routes/interviews.js	schedule, list, update
/api/feedback	routes/feedback.js	submit, get all
/api/dashboard	routes/dashboard.js	stats, activity
/api/ai	routes/ai.js	resume scoring, GenAI report
/api/ml	routes/ml.js	selection prediction
/api/notifications	routes/notifications.js	get, read, clear

3.5 Authentication Flow

1. User submits email and password to POST /api/auth/login
2. Backend fetches user from MySQL and compares password using bcrypt.compare()
3. On success, JWT token generated with user id, name, email, and role
4. Token stored in browser localStorage by the frontend
5. Every subsequent API request includes Authorization: Bearer token header
6. JWT middleware on backend verifies token before processing request

4. Tools Used

4.1 Frontend Technologies

Tool	Version	Purpose
HTML5	Latest	Page structure and markup
CSS3	Latest	Styling with custom design system
Vanilla JavaScript	ES2022	DOM manipulation and API calls
Chart.js	Latest	Analytics charts
FullCalendar.js	3.10.2	Interview calendar view
Font Awesome	6.4.2	Icons
Google Fonts	DM Sans, Syne	Typography

4.2 Backend Technologies

Tool	Version	Purpose
Node.js	v20	JavaScript runtime
Express.js	4.19	REST API framework
mysql2	3.9	MySQL database driver
bcrypt	5.1	Password hashing
jsonwebtoken	9.0	JWT generation and verification
multer	1.4	File upload handling
cors	2.8	Cross-Origin Resource Sharing
dotenv	16.4	Environment variable management

4.3 Machine Learning Tools

Tool	Version	Purpose
Python	3.9+	ML programming language
scikit-learn	Latest	Random Forest implementation
pandas	Latest	Dataset preprocessing
numpy	Latest	Numerical computations
matplotlib	Latest	Visualization
seaborn	Latest	Statistical visualization
joblib	Latest	Model serialization to .pkl
Jupyter Notebook	Latest	ML development environment

4.4 AI and External APIs

Tool	Details	Purpose
Groq API	Free tier	LLM API provider
Meta Llama 3.1	8B parameter model	Natural language report generation

4.5 Database

Tool	Version	Purpose
MySQL	8.0	Primary relational database
MySQL Workbench	8.0	Database design and management
Railway MySQL	Managed	Cloud database hosting

4.6 Development and Deployment Tools

Tool	Purpose
VS Code	Primary code editor
Git	Version control
GitHub	Source code repository and CI/CD
Render	Frontend and backend hosting
Railway	Managed MySQL database hosting
Nodemon	Auto-restart during development
Postman	API testing

5. Deployment Details

Service	Platform	URL
Frontend	Render	https://interviewpro-ats.onrender.com
Backend API	Render	https://ai-interview-management-system.onrender.com
MySQL Database	Railway	Private internal URL
Source Code	GitHub	github.com/BVarunReddy/ai-interview-management-system

6. Conclusion

InterviewPro successfully demonstrates the integration of modern web development, machine learning, and generative AI in solving a real-world recruitment management problem. The system provides a complete end-to-end recruitment workflow that replaces manual processes with automated, data-driven workflows.

The Random Forest model achieves 90%+ accuracy in predicting candidate selection probability, while the Groq Llama 3.1 LLM generates professional, unique evaluation reports. Together, these AI capabilities significantly enhance recruiter productivity and decision quality.

The project is fully deployed and accessible, demonstrating real-world production deployment skills including cloud hosting, database management, API integration, and CI/CD through GitHub auto-deploy.